

Implementation-Pipeline Audit for OccuFly V-JEPA Depth and Semantic Scene Completion Experiments

Pipeline trace generated from local experiment outputs and source code

July 2, 2026

1 Scope and Provenance

This document traces the implementation pipelines that produced the requested result folders and summarizes them in paper-ready architectural language. The inspected code paths include the supervised depth scripts in `experiments_related/scripts`, the video-depth projects in `experiments_related/vjepa21video_scripts`, the CGFormer comparison package, and the V-JEPA/FoundationSSC semantic scene completion packages in `experiments_related/scripts_ssc`. All tensor sizes below refer to the actual OccuFly configuration used in the saved run configurations unless stated otherwise.

Result folders covered.

- Linear depth probes: `vjepa21_dinov3_linearprobe_occufly_maskaware_rerun_ddp` and `vjepa21video_features_depth/real_stlp_clip4`.
- DPT-FiLM depth probes: `vjepa21_vitG_dpt_film_occufly_sweep`, `vjepa21_vitG_final_sfp_dpt_film_occufly_sweep`, `vjepa21video_features_depth/dptfilm_attention`, and `30_50train_40_1_4_val_40_5_8_test`.
- Semantic scene completion: `ssc/cgformer_results_comparision`, `ssc/vjepaF2Dfoundationssc_foundationssc_hybrid_original/...`, `ssc/vjepaf2Dfoundationssc_mde_warmstart_G`, `ssc/vjepafoundationssc_foundationssc_hybrid_original_geometry_fix_sm90_corrected_8gpu_20260630_015221`, and `ssc/vjepafoundationssc_foundationssc_hybrid_original_semantic_reweighting_only_fresh_8gpu_20260701_080802`.

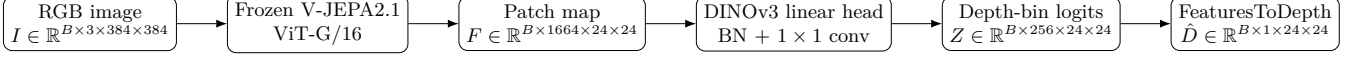
Common OccuFly protocol. For monocular depth, the standard scene split is train `{scene_01, ..., scene_05}`, validation `{scene_06, scene_07}`, and test `{scene_08, scene_09}`, over altitudes `{30, 40, 50}` m. The canonical image resolution is 384×384 , patch size is 16, V-JEPA2.1 ViT-Gigantic has $C = 1664$ channels and 48 transformer blocks, and the spatial patch grid is 24×24 . Depth is metric and clipped/evaluated in `[0.001, 80]` m. Invalid depth pixels are finite-masked before resizing; loss and metrics use only valid pixels.

2 Frozen V-JEPA2.1/DINOv3 Mask-Aware Linear Depth Probe

2.1 High-Level Overview

This pipeline is a strict frozen-backbone linear probe for monocular metric depth. RGB images are encoded by a frozen V-JEPA2.1 ViT-Gigantic backbone, the final dense patch map is passed to a DINOv3-style `SyncBatchNorm + Conv2d(1 × 1)` linear head, and the 256-channel output is converted to metric depth by `FeaturesToDepth`. The result folder `vjepa21_dinov3_linearprobe_occufly_maskaware_rerun_ddp` was trained with distributed data parallelism, global batch size 8, 20 epochs, and LR/WD sweeps. The inspected completed point `1r_1p5e_3_wd_1e_3` uses 14,804 training samples, 1,965 validation samples, and 3,842 test samples.

Implementation paths. The core implementation is `experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py`. The DDP sweep/resume launcher is `experiments_related/scripts/linearprobe/run_maskaware_occufly_ddp_sweep_resume.sh`, and the altitude/BatchNorm probe variants are documented and launched from `experiments_related/scripts/linearprobe/README.md` and related `experiments_related/scripts/linearprobe/train_occufly_dino_linear_bn_*.py` scripts. The V-JEPA/DINO bridge used by this family is `experiments_related/scripts/vjepa21_dinov3_depth_adapter.py`.



2.2 Mathematical Formulation

Let $I_b \in \mathbb{R}^{3 \times 384 \times 384}$. The frozen encoder E_θ^{VJ} returns a patch feature tensor

$$F_b = E_\theta^{\text{VJ}}(I_b) \in \mathbb{R}^{1664 \times 24 \times 24}, \quad \theta \text{ frozen.}$$

The probe is a per-location affine classifier over depth bins:

$$Z_b(:, u, v) = W \text{BN}(F_b(:, u, v)) + a, \quad W \in \mathbb{R}^{256 \times 1664}.$$

For linear-bin conversion, with d_k uniformly spaced in $[d_{\min}, d_{\max}]$, **FeaturesToDepth** computes normalized nonnegative bin weights $p_{k,u,v}$ and

$$\hat{D}_{b,u,v} = \sum_{k=1}^{256} p_{b,k,u,v} d_k.$$

The ground-truth depth D and validity mask M are resized using weighted mask-aware interpolation:

$$D' = \frac{\text{resize}(D \odot M)}{\max(\text{resize}(M), \epsilon)}, \quad M' = \text{resize}(M) > \tau.$$

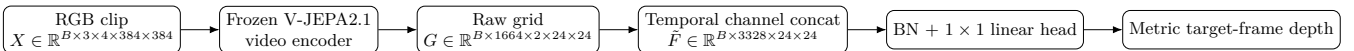
The training criterion is masked scale-invariant depth loss (**SIGLOSS**) and metrics include RMSE, AbsRel, and log-RMSE over M' .

3 V-JEPA2.1 Video Spatio-Temporal Linear Probe (ST-LP)

3.1 Definition of ST-LP

ST-LP means **Spatio-Temporal Linear Probe**. It is the video-mode analogue of the image linear depth probe, but it preserves the V-JEPA video tubelet axis instead of averaging it. The folder `vjepa21video_features_depth/real_stlp_clip4` uses target-centered real clips with four RGB frames. The backbone is frozen, selected layer is the final V-JEPA layer 47, and the saved run reports 858,880 trainable decoder parameters.

Implementation paths. The video ST-LP project lives under `experiments_related/vjepa21video_scripts`. The main trainer is `experiments_related/vjepa21video_scripts/scripts/train_stlp_depth.py`; the sequential ablation runner is `experiments_related/vjepa21video_scripts/scripts/run_all_stlp_experiments.py`; and the implementation modules are `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/depth_heads.py`, `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/vjepa_backbone.py`, `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/ocufly_data.py`, and `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/losses_metrics.py`.



3.2 Inputs, Outputs, and Ablation Meaning

For a 4-frame clip and V-JEPA tubelet size 2, $T' = 4/2 = 2$. The encoder output is

$$G = E_\theta^{\text{video}}(X) \in \mathbb{R}^{B \times C \times T' \times H' \times W'} = \mathbb{R}^{B \times 1664 \times 2 \times 24 \times 24}.$$

ST-LP reorders the tensor by

$$\tilde{F}_{b,(tC+c),u,v} = G_{b,c,t,u,v}, \quad \tilde{F} \in \mathbb{R}^{B \times (T'C) \times H' \times W'} = \mathbb{R}^{B \times 3328 \times 24 \times 24}.$$

The linear head weight tensor is $W \in \mathbb{R}^{256 \times 3328 \times 1 \times 1}$, and temporal importance can be recovered by reshaping $W \rightarrow \mathbb{R}^{256 \times T' \times C}$ and summing absolute mass per tubelet. The three intended video ablations are: **repeated_stlp**, where the same target frame is repeated; **real_stlp**, where the target-centered temporal context is real; and **shuffled_stlp**, where real context frames are deterministically shuffled. Missing target depth and first/last frames are excluded from training and evaluation.

4 Image-Mode V-JEPA2.1 DPT-FiLM Depth Decoder

4.1 High-Level Overview

The DPT-FiLM pipeline replaces the linear head with a multi-layer DPT decoder and metadata conditioning. The folder `vjepa21_vitG_dpt_film_occufly_sweep` uses selected V-JEPA transformer layers $\{11, 23, 37, 47\}$, each reshaped to a 24×24 patch map, then passed through DINOv3 DPT reassembly/projection, FiLM metadata modulation, DPT fusion/refinement, and depth-bin conversion.

Implementation paths. The image-mode DPT-FiLM trainer is `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py`. Its run notes and command templates are in `experiments_related/scripts/dpt_decoder/README.md`. The implementation reuses the DINOv3 depth-head adapter and V-JEPA checkpoint loading utilities from `experiments_related/scripts/vjepa21_dinov3_depth_adapter.py`.

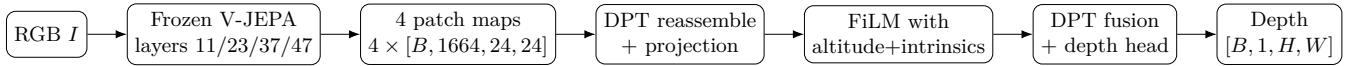
FiLM is Feature-wise Linear Modulation. The metadata vector is

$$m = [h_{\text{norm}}, f_x/W, f_y/H, c_x/W - 0.5, c_y/H - 0.5] \in \mathbb{R}^5, \quad h_{\text{norm}} = (h - 40)/10.$$

A metadata MLP maps m to $e_m \in \mathbb{R}^{128}$. Each DPT level l applies

$$\text{FiLM}_l(F_l, e_m) = F_l \odot (1 + \gamma_l(e_m)) + \beta_l(e_m),$$

where $\gamma_l, \beta_l \in \mathbb{R}^{C_d}$ are predicted by a linear layer initialized to identity behavior. In the inspected run configs, $C_d = 256$, hidden channels are 32, and DPT post-process channels are $[128, 256, 512, 1024]$.



4.2 Final-Layer SFP DPT-FiLM Variant

The folder `vjepa21_vitG_final_sfp_dpt_film_occufly_sweep` uses the same depth target and FiLM conditioning but changes the feature source. Instead of requesting four intermediate transformer layers, it takes the final V-JEPA token sequence only, reshapes it to $F^{47} \in \mathbb{R}^{B \times 1664 \times 24 \times 24}$, and expands it with a local **SFP (Simple Feature Pyramid)** adapted from ViTDet/Detectron2:

$$\text{SFP}(F^{47}) = \{F_4, F_2, F_1, F_{1/2}\}, \quad F_s \in \mathbb{R}^{B \times 256 \times (24s) \times (24s)}, \quad s \in \{4, 2, 1, 1/2\}.$$

Thus the four DPT levels are approximately 96×96 , 48×48 , 24×24 , and 12×12 , each with 256 channels. The DPT post-process channels in this variant are $[256, 256, 256, 256]$, and the run uses frozen V-JEPA, distributed training, 5 GPUs for the inspected point, and the same metric-depth conversion.

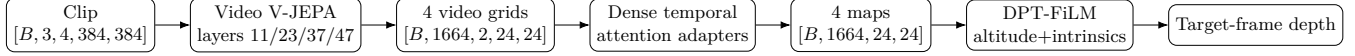
Implementation paths. The final-layer SFP variant is implemented in `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_final_sfp_dpt_film_occufly.py`. The same README, `experiments_related/scripts/dpt_decoder/README.md`, records the intended command line and explicitly distinguishes this final-token SFP path from the four-intermediate-layer DPT-FiLM path.

5 Video DPT-FiLM with Dense Temporal Attention (DTA)

5.1 High-Level Overview

The folder `vjepa21video_features_depth/dptfilm_attention` implements video-mode DPT-FiLM. Its `real_dta_clip4/lr_1e-3_wd_1e-3` run uses four-frame clips, selected layers $\{11, 23, 37, 47\}$, frozen V-JEPA2.1-Gigantic, 30 epochs, $\text{lr} = 10^{-3}$, $\text{wd} = 10^{-3}$, metadata dimension 128, and 8 temporal attention heads. **DTA** means **Dense Temporal Attention**: at each spatial cell and selected layer, a learned query attends over the tubelet features.

Implementation paths. The video DPT-FiLM+DTA project lives at `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention`. The main trainer is `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/scripts/train_video_dptfilm_attention.py`; the sequential runner is `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/scripts/run_all_dptfilm_attention_experiments.py`; and the relevant model modules are `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/model.py`, `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/temporal_attention.py`, `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/dpt_film.py`, and `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/vjepa21_video_adapter.py`.



5.2 Dense Temporal Attention Notation

For each selected layer l , the video backbone returns $G_l \in \mathbb{R}^{B \times 1664 \times T' \times 24 \times 24}$, with $T' = 2$ for 4-frame clips and $T' = 4$ for 8-frame clips. At every spatial site (u, v) , the adapter forms a temporal token sequence

$$S_{b, :, u, v}^{(l)} = [G_{b, :, 1, u, v}^{(l)}, \dots, G_{b, :, T', u, v}^{(l)}] \in \mathbb{R}^{T' \times 1664}.$$

A $1 \times 1 \times 1$ projection reduces channels to $A = 512$, a learned query $q \in \mathbb{R}^{1 \times A}$ attends over the T' tokens with 8-head multi-head attention, and a residual interpolation with initial scale 0.1 yields

$$\bar{F}_{b, :, u, v}^{(l)} = \text{mean}_t(G_{b, :, t, u, v}^{(l)}) + \alpha \left(\text{Expand}(\text{MHA}(q, K, V)) - \text{mean}_t(G_{b, :, t, u, v}^{(l)}) \right).$$

The adapted maps $\{\bar{F}^{(l)}\}$ are consumed by the same DPT-FiLM decoder as the image-mode four-layer variant. The available tags are `repeated_dta`, `real_dta`, and `shuffled_dta`.

6 Altitude-Split DPT-FiLM Depth Protocol

The folder `30_50train_40_1_4_val_40_5_8_test` uses the same DPT-FiLM training code path but a different generalization protocol: train on 30 m and 50 m data, validate on 40 m scenes 01–04, and test on 40 m scenes 05–08. This is not merely a random scene split; it explicitly tests whether the learned depth model interpolates to the unseen middle altitude. The saved sweep contains LR/WD points and per-altitude metrics. In notation, the training set is

$$\mathcal{D}_{\text{train}} = \{(I, D, m) : h \in \{30, 50\}\}, \quad \mathcal{D}_{\text{val/test}} \subset \{(I, D, m) : h = 40\},$$

with the same metadata definition $m = [h_{\text{norm}}, f_x/W, f_y/H, c_x/W - 0.5, c_y/H - 0.5]$. This folder should be reported as an altitude-interpolation stress test for the DPT-FiLM depth decoder.

Implementation paths. This split is controlled by the split-protocol support in `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py`. The launcher/logs in the output folder point back to that trainer with a non-default split protocol that trains on altitudes 30 and 50, validates on 40 m scenes 01–04, and tests on 40 m scenes 05–08.

7 CGFormer-Based Semantic Scene Completion Comparison

7.1 High-Level Overview

The folder `ssc/cgformer_results_comparison` is an aggregate comparison of six CGFormer/OccuFly SSC runs: `dav2_prior`, `m2_vjepa_distill_projector_current`, `m3_latent_view`, `m4_masked3d_projector_current`, and two V-JEPA2.1 ViT-G DPT-FiLM depth-prior runs. It is a reporting layer, not a new model: `runs_wide.csv`, `metrics_long.csv`, `leaderboard_long.csv`, `altitude_metrics_long.csv`, and `class_metrics_long.csv` are generated from each run's `results_ssc.csv` and `metrics_test.json`.

Implementation paths. The comparison dashboard and table generator in the result folder are `experiments_related/outputs/results/ssc/cgformer_results_comparision/view_results.py` and `experiments_related/outputs/results/ssc/cgformer_results_comparision/README.md`. The CGFormer OccuFly preparation/evaluation utilities are under `experiments_related/scripts_ssc/cgformer/common`, with V-JEPA/DPT-FiLM prior export and launch scripts under `experiments_related/scripts_ssc/cgformer/vitGFiLMaltitudeconditioning`. The local baseline model repository is CGFormer.

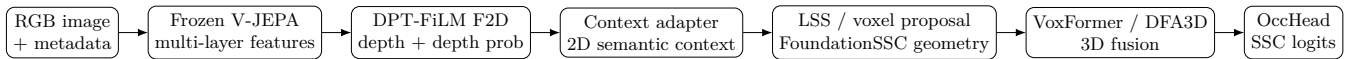
CGFormer is used as a semantic scene completion baseline family with image-to-BEV/voxel modules from the local CGFormer codebase. The comparison emphasizes occupied-class mIoU, occupied IoU, class-wise IoU, altitude-stratified metrics, and deltas against the DAV2-prior baseline. DAV2 means Depth Anything V2; V-JEPA means Video Joint Embedding Predictive Architecture; BEV means Bird’s-Eye View; SSC means Semantic Scene Completion.

8 V-JEPA/F2D + FoundationSSC Hybrid SSC

8.1 Architecture Overview

The requested `vjepaf2dfoundationssc_foundationssc_hybrid_original` and corrected `vjepafoundationssc_..` folders implement a hybrid semantic scene completion system. The front end is V-JEPA2.1 plus DPT-FiLM depth/probability estimation; the 3D back end is real FoundationSSC/MMCV/MMDetection/DFA3D with OccuFly geometry patched into its config before module construction.

Implementation paths. The active corrected FoundationSSC hybrid source is `experiments_related/scripts_ssc/vjepafoundationssc_foundationssc_hybrid_original`. The main trainer is `experiments_related/scripts_ssc/vjepafoundationssc_foundationssc_hybrid_original/vjepafoundationssc/train_vjepa_occufly_ssc.py`; the depth front end is `experiments_related/scripts_ssc/vjepafoundationssc_foundationssc_hybrid_original/vjepafoundationssc/dpt_film_depth_probability_head.py`; the V-JEPA backbone wrapper is `experiments_related/scripts_ssc/vjepafoundationssc_foundationssc_hybrid_original/vjepafoundationssc/backbone_factory.py`; the lifting code is `experiments_related/scripts_ssc/vjepafoundationssc_foundationssc_hybrid_original/vjepafoundationssc/frustum_voxel_pool.py`; losses and metrics are `experiments_related/scripts_ssc/vjepafoundationssc_foundationssc_hybrid_original/vjepafoundationssc/losses_ssc.py` and `experiments_related/scripts_ssc/vjepafoundationssc_foundationssc_hybrid_original/vjepafoundationssc/metrics_ssc.py`; and the real FoundationSSC adapters are in `experiments_related/scripts_ssc/vjepafoundationssc_foundationssc_hybrid_original/vjepafoundationssc/foundation_ssc/real_adapters.py`. The older F2D package variant is under `experiments_related/scripts_ssc/vjepaf2dfoundationssc_foundationssc_hybrid_original`.



F2D is used here as **features-to-depth**: DPT-FiLM predicts both dense metric depth $\hat{D}$ and a depth-bin probability tensor P_D . **LSS** is **Lift-Splat-Shoot**; this is the camera-to-voxel lifting stage. The user note asking whether `vjepaf2dfoundationssc_mde_warmstart_g` was a “Look Spat Shoot” baseline appears to refer to **Lift-Splat-Shoot**, not “Look Spat Shoot”. Based on the local code and logs, `vjepaf2dfoundationssc_mde_warmstart_g` is better described as a V-JEPA/FoundationSSC model warm-started from a monocular depth estimation (MDE) checkpoint, using LSS-style depth-probability lifting inside the SSC model.

8.2 OccuFly SSC Tensor Geometry

OccuFly semantic labels are stored in WHD layout and converted internally to DHW:

$$Y_{\text{WHD}} \in \{0, \dots, 36, 255\}^{192 \times 128 \times 128}, \quad Y_{\text{DHW}} \in \{0, \dots, 36, 255\}^{128 \times 128 \times 192}.$$

The full voxel grid spans

$$x \in [-48, 48] \text{ m}, \quad y \in [-32, 32] \text{ m}, \quad z \in [0, 64] \text{ m}, \quad \Delta = 0.5 \text{ m},$$

with 37 semantic classes, class 0 as empty/free space, and 255 as ignore/unknown. With voxel downsample (2, 2, 2), the FoundationSSC low-resolution grid is

$$V_{\text{low}} = (96, 64, 64)_{\text{WHD}},$$

and runtime diagnostics reported

$$\begin{aligned} &\text{proposal } [B, 1, 96, 64, 64], \quad \text{LSS volume } [B, 128, 96, 64, 64], \quad \text{VoxFormer volume } [B, 128, 96, 64, 64], \\ &\text{logits } [B, 37, 128, 128, 192], \quad \text{label } [B, 128, 128, 192]. \end{aligned}$$

The geometry-fix run rewrites FoundationSSC’s stale SemanticKITTI geometry [128, 128, 16] to OccuFly-compatible bounds and asserts that stale shapes do not reappear.

8.3 Lifting and Depth Probability

For a feature map $C \in \mathbb{R}^{B \times C_c \times H_f \times W_f}$ and depth probability $P_D \in \mathbb{R}^{B \times K \times H_f \times W_f}$, the frustum feature is

$$\Phi_{b,c,k,u,v} = C_{b,c,u,v} P_{D,b,k,u,v}.$$

Using intrinsics (f_x, f_y, c_x, c_y) , pixel (u, v) , and bin center d_k ,

$$x = \frac{(u - c_x)d_k}{f_x}, \quad y = \frac{(v - c_y)d_k}{f_y}, \quad z = d_k,$$

and the feature is accumulated into voxel index

$$i_x = \left\lfloor \frac{x - x_{\min}}{\Delta_x} \right\rfloor, \quad i_y = \left\lfloor \frac{y - y_{\min}}{\Delta_y} \right\rfloor, \quad i_z = \left\lfloor \frac{z - z_{\min}}{\Delta_z} \right\rfloor.$$

The corrected FoundationSSC path remaps DPT-FiLM depth probabilities to FoundationSSC dbound centers and preserves the dense-depth expectation; diagnostics measured a mean absolute expectation error of approximately 1.76×10^{-6} m.

8.4 Losses and Variants

The SSC objective combines semantic cross entropy, semantic scaling loss, geometric scaling loss, optional occupancy BCE, and depth supervision:

$$\mathcal{L} = \lambda_{\text{ce}} \mathcal{L}_{\text{ssc-ce}} + \lambda_{\text{sem}} \mathcal{L}_{\text{sem-scal}} + \lambda_{\text{geo}} \mathcal{L}_{\text{geo-scal}} + \lambda_{\text{occ}} \mathcal{L}_{\text{occ}} + \lambda_{\text{dce}} \mathcal{L}_{\text{depth-ce}} + \lambda_{\text{dl1}} \mathcal{L}_{\text{depth-l1}}.$$

The inspected corrected defaults are $\lambda_{\text{ce}} = \lambda_{\text{sem}} = \lambda_{\text{geo}} = 1$, $\lambda_{\text{occ}} = 0$, $\lambda_{\text{dce}} = 0.5$, and $\lambda_{\text{dl1}} = 0.2$. Thus occupancy loss may be logged but does not affect optimization unless enabled.

MDE warm-start run. `vjepafoundationssc_mde_warmstart_g` initializes the SSC depth head from a monocular depth estimation checkpoint. The code path loads the MDE checkpoint into the frozen depth head only and does not load it into the context DPT branch. It is an MDE-warm-started LSS/FoundationSSC hybrid, not a separate “Look Spat Shoot” baseline.

Geometry-fix run. `vjepafoundationssc_foundationssc_hybrid_original_geometry_fix_sm90_corrected_8gpu_20260630_015221` is the corrected real-FoundationSSC run. Its key methodological contribution is the construction-time geometry rewrite to OccuFly bounds and tensor sizes before LSS, proposal, VoxFormer, and OccHead modules are built.

Semantic-reweighting-only fresh run. `vjepafoundationssc_foundationssc_hybrid_original_semantic_reweighting_only_fresh_8gpu_20260701_080802` is a fresh 8-GPU run focused on semantic class/loss reweighting while preserving the corrected FoundationSSC geometry. This should be reported as a loss-balancing ablation over the geometry-corrected architecture, not as a new feature-extraction architecture.

9 Abbreviations

Term	Expansion / meaning
AbsRel	Absolute relative error
BEV	Bird’s-eye view
BN	Batch normalization
CGFormer	Camera/geometry transformer baseline for semantic scene completion
DAV2	Depth Anything V2
DDP	Distributed data parallelism
DFA3D	Deformable feature aggregation 3D extension used by FoundationSSC
DHW / WHD	Depth-height-width / width-height-depth voxel tensor order
DINOv3	Self-supervised visual foundation model family used for depth heads
DPT	Dense Prediction Transformer
DTA	Dense Temporal Attention
F2D	Features-to-depth in this codebase: dense depth and depth probability frontend
FiLM	Feature-wise Linear Modulation
FoundationSSC	Foundation-model semantic scene completion architecture/package
LSS	Lift-Splat-Shoot, camera frustum lifting into a voxel/BEV representation
MDE	Monocular depth estimation
OccHead	Occupancy/semantic voxel prediction head
RMSE	Root mean squared error
SFP	Simple Feature Pyramid
SIGLOSS	Scale-invariant logarithmic depth loss
SSC	Semantic Scene Completion
ST-LP	Spatio-Temporal Linear Probe
V-JEPA	Video Joint Embedding Predictive Architecture
ViT-G	Vision Transformer Gigantic variant
VoxFormer	Voxel transformer module used in FoundationSSC-style 3D reasoning

10 Reporting Recommendations

For a research paper, these experiments should be grouped into three methodological families: (1) frozen V-JEPA representation probing for metric depth, covering image LP and video ST-LP; (2) structured depth decoding with DPT-FiLM and temporal attention, covering image DPT-FiLM, final-layer SFP DPT-FiLM, altitude-interpolation DPT-FiLM, and video DTA; and (3) semantic scene completion, covering CGFormer baselines, V-JEPA/F2D/FoundationSSC hybrids, MDE warm-starting, geometry correction, and semantic reweighting. The cleanest notation is to present depth as I or $X \rightarrow \hat{D}$, and SSC as $(I, m, K) \rightarrow \hat{Y}_{DHW}$, where m is altitude/intrinsics metadata and K denotes camera calibration.